Rubric-Oriented Architecture Report

Problem Understanding

The challenge addresses governance and context management in AI-assisted development: Git tracks what and when, not why (intent) or structural identity (AST). The system must (1) bind intent to code via a machine-readable trace, (2) enforce context before action, (3) use a clean hook middleware, and (4) support parallel agents without collision. The implementation treats the codebase as a set of formalized intents (active_intents.yaml); every mutating write is gated by intent and scope and recorded with intent_id and content_hash. Refactors vs features are distinguished by mutation_class (AST_REFACTOR | INTENT_EVOLUTION). Context is injected per intent only, not dumped. Parallel panels share .orchestration/ and avoid overwrites via optimistic locking (read-hash vs current file hash).

Architectural Design Quality

Layers: Webview (UI only, postMessage), Extension Host (Task, tool dispatch, API), Hook Engine (middleware: context, gate, trace). The execution loop in presentAssistantMessage does not contain hook logic; tools call the hook layer. Hook implementation is split into HookManager (orchestrator), ContextLayer (intent context build), IntentPipeline (write gate), CorrelationService (trace append), and lifecycle types. **Input/output:** Each hook phase has a defined input (e.g. cwd, intentId or task, relPath, args) and output (PreHookResult / PostHookResult). **Failure handling:** Pre-hooks return allowed: false and message; tools push the message and abort. No exceptions for business rules. **Invariants:** Trace lines are valid AgentTraceRecord with intent_id and mutation_class; every trace write corresponds to a prior successful preWriteFile gate; content_hash is deterministic (SHA-256 prefix).

Hook System Design Excellence

Pattern: Middleware/Interceptor. Tools invoke the hook manager before and after the operation; the host’s tool dispatch is unchanged. **Isolation:** Hook code lives under src/hooks/; no orchestration logic in presentAssistantMessage or in tool implementations beyond the call to hookEngine. **Composability:** ContextLayer, IntentPipeline, and CorrelationService are separate modules; HookManager composes them. **Fail-safe:** Hooks return results; they do not throw for validation failures. **Lifecycle:** HookLifecycle.ts defines HookPhase (Pre/Post) and HookContract; the current flow uses HookManager methods that map to these phases. **Contract:** Pre-hook returns { allowed, message?, injectedContext? }; post-hook returns { success, message? }. Tools respect the contract (abort on !allowed; append trace only after successful write).

Intent-Code Traceability Rigor

Mapping: agent_trace.jsonl stores one JSON object per write. Each record has intent_id, mutation_class, and files[].conversations[].ranges[].content_hash. The relation IntentId × (FilePath, ContentHash, MutationClass, Timestamp) is machine-readable and queryable. **Content hash:** SHA-256 of content (UTF-8), prefix "sha256:" + 32 hex chars. Deterministic and spatially independent. **Refactor vs feature:** mutation_class AST_REFACTOR = behavior-preserving change under the same intent; INTENT_EVOLUTION = new behavior or feature. Stored per record; analytics can filter by class. **Golden thread:** intent_id in

active_intents → selected by task (select_active_intent) → passed or implied in write_to_file → stored in record.intent_id and in files[].conversations[].related (type "specification", value intent_id).

Agent & Context Engineering

Context DB: active_intents.yaml is the source. The agent does not receive the full file by default. **Reference before act:** The system prompt includes an intent protocol section requiring select_active_intent before any write when .orchestration/ exists. preWriteFile requires intent_id (from task or from write_to_file args). So the agent must have referenced the context (by calling select_active_intent or supplying intent_id) before the write is allowed. **Curated injection:** getIntentContext returns only the selected intent's id, name, status, owned_scope, constraints, acceptance_criteria in XML. No dump. **Dynamic state:** Hooks read active_intents and .intentignore on each call; no long-lived cache. Trace and CLAUDE.md are updated by post-hooks and record_lesson. **Shared brain:** CLAUDE.md is appended by record_lesson; agents read it via read_file when needed. No automatic injection into every prompt to avoid context bloat.

Code Quality & Modularity

Modularity: hooks/ contains HookEngine (facade), HookManager (orchestrator), lifecycle (phase/contract types), context (ContextLayer), correlation (CorrelationService), pipeline (IntentPipeline), plus orchestration-io, content-hash, scope-match, command-classify. Tools depend only on HookEngine. **Dependency direction:** core/tools → hooks; hooks → core/task only for Task type and cwd. **Single responsibility:** ContextLayer builds context; CorrelationService appends trace; IntentPipeline validates write gate; HookManager coordinates and holds task state. **No monolith:** Hook logic is not embedded in the tool dispatch switch; it is in dedicated modules.

Innovation & Scalability

Extensibility: HookLifecycle defines a contract (phase, toolName, run). A future hook registry could register handlers per (phase, toolName) and HookManager could invoke them without changing tools. **Parallel orchestration:** Multiple tasks (chat panels) share .orchestration/. Collision avoidance: optimistic locking—read-hash stored when the tool reads the file for diff; preWriteFile compares current file hash to that; on mismatch, write blocked with "Stale File". **Scalability:** agent_trace.jsonl is append-only; queries are linear scan. For very large histories, an indexed store or archival could be added. Intent count scales with YAML size; context injection is per intent so token usage stays bounded.

Technical Trade-offs

- **Context injection vs full dump:** Single-intent injection reduces tokens and drift; the model does not see other intents unless it selects or reads. Trade-off: explicit pull for full catalog when needed.
- **Pre/post inside tools vs wrapping dispatch:** Calling the hook from inside the tools that need it keeps the dispatch loop clean and avoids a central hook registry that knows every tool name. Trade-off: new mutating tools must be wired to the hook for traceability.
- **File-based persistence:** .orchestration/ as YAML/JSONL/MD keeps the implementation simple and portable. Trade-off: no indexing; large trace requires scan or future migration to a DB.
- **Read-hash only on write path:** Optimistic lock uses the hash set when preparing the diff. read_file does not set it. Trade-off: locking covers read-then-write in one flow; full read tracking would add state and complexity.